Control Panel of Morse Beacon- up to 26 points + 10 premium points

Version 2.0 – *Sept 14, 2025*

Table of Contents

Assignment....1

Usable Clocks....2

Steps to your points....3

A: Blinking - from +1 point, aka credit emergency, to +6 points....3

B: +5 points - Morse dot-dash code....3

C: +5 points - Morse dot-dash with letters....4

D: +5 points – Control Morse speed....4

E: +5 points -Control waiting time....4

F: +10 extra points – Text path....4

Appendix: IRDA....6

Assignment

Convert your LCD background to the control panel for your Morse beacon.

Requirements:

- Of course, your solution should fit into the FPGA and work properly. It must also successfully meet the requirements of the **Quartus Checker from LSP Tools** that will detect LATCH, LOOP, Dividers, and Modulo operations with numbers different than powers of 2.
- TimeQuest Analyzer can only report "Unconstrained Paths" and nothing else.
- Your solution must not utilize any elements on the **front part of the Veek-MT2 board**, e.g., LEDR, LEDG, SW, KEY, HEX*, and so on. The control panel contains only the LCD and nothing else. Use a touch and IRDA remote control.
- Again, the **ieee.numeric_std** library is only allowed for arithmetic operations.
- The project must not use shared **variables**. These were introduced for simulations only.

Otherwise, solve the task as you please**.**

Sample code – you can load the example of how to use the IRDA Module and the I2C Touch Module. However, the last task is your independent work. There is no sample solution or a guide.

Why not let students design IRDA and Touch modules?

In modern times, we rarely start from scratch. For example, we don't write a module to communicate over the network in classical programming languages, but use ready-made network sockets.

Yes, we can provide students with an overview of I2C bus signal flows and IRDA standards and let them design their own drivers, but it will cost them a lot of time thinking about solutions to tasks that have already been solved :-)

Even assembling a circuit from ready-made blocks will require a lot of work. You have received VeekMT2_LCDgenV2.vhd for the LCD background task for the same reasons.

Usable Clocks

VeekMT2_I2CTouchLCD uses the CLOCK_50 clock (50 MHZ), but it could create metastability in LCD logic that depends on LCD_DCLK (33 MHz).- In other words, TimeQuest Analyzer shows red complaints about your designed circuit. Although it may work by chance now, it may soon stop, only to start working again after an indefinite period, as if by magic. ☺

Question: Can CLOCK_50 be used in LCD logic without TimeQuest complaining?

Yes. The VeekMT2_I2CTouchLCD accepts data from bus I2C synchronized with CLOCK50 and transforms its output to the LCD_DCLK clock domain by synchronizer chains.

The easiest way for your logic is timing based on **signals derived from LCD_DCLK**:

- **LCD_DCLK** - **33.000 MHZ**, 50% duty cycle.
- **XEND_N** - pulse '0' when xcolumn=1023; otherwise, it is in '1'. It signals the last column of a row.
  Frequency: **32.2 kHz** = 33 MHz/1024.
- **YEND_N** - pulse '0' at yrow=524, otherwise in '1'. It signals the last line of a frame.
  Frequency: **61.4 Hz** =33 MHz/(1024*525).

Steps to your points

The task offers five different levels of complexity. It is logically necessary to climb through all its lower floors to reach a higher level, i.e., to meet all the previous requirements, even for 1 point.

Remember that every step has a straightforward implementation.
If your solution grows too complex, you have chosen the wrong approach !!!

A: Blinking - from +1 point, aka credit emergency, to +6 points

Choose an element that will simulate a lamp gradually lighting up in rhythm with your Morse code from your Beacon task. **The Morse blinking can be started or stopped by touching the LCD.**

- +1 point is given if the element has no animation, e.g., it is just switched on/off. Everyone can solve this quickly if other courses take up all the time. ☹
- +6 points - If the element changes its color and size depending on how long it has been lit, it creates the impression of a lamp. In hardware, we can achieve dynamic effects relatively cheaply.

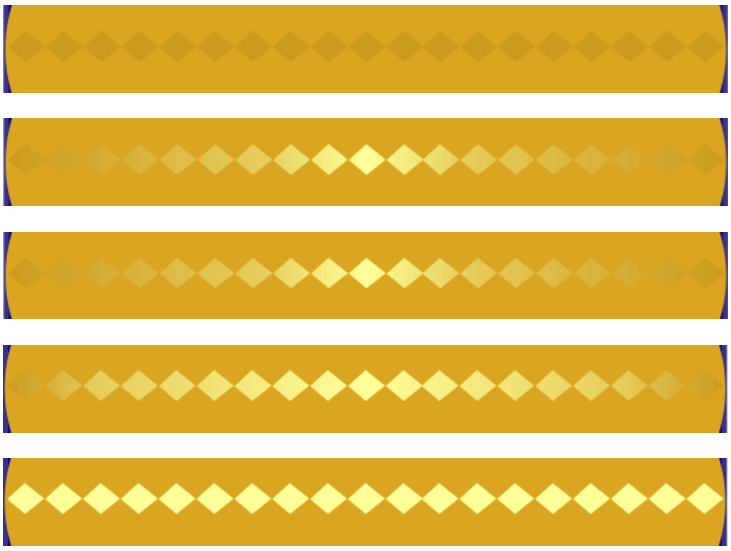


Figure 1 - One of many possible dynamic animations

You can add a generic parameter that suppresses this animation in the case of GHDL simulations to speed them up, and let it only for the Veek-MT2 board.

B: +5 points - Morse dot-dash code

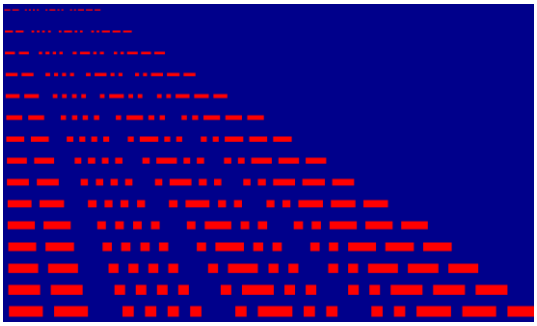
The animation of the running dot-comma code is added to the blinking element from step A; see the images in the next step. The Morse dot-dash code should be enlarged at least to 90 % of the LCD width.

Do not use complex expressions; it is a simple 1-D linear transformation!

The x coordinate of LCD in the range from 0 to 799 is mapped to ixMorse. In the case of MHL2, ixMorse ranges from 0 to 49. We calculate

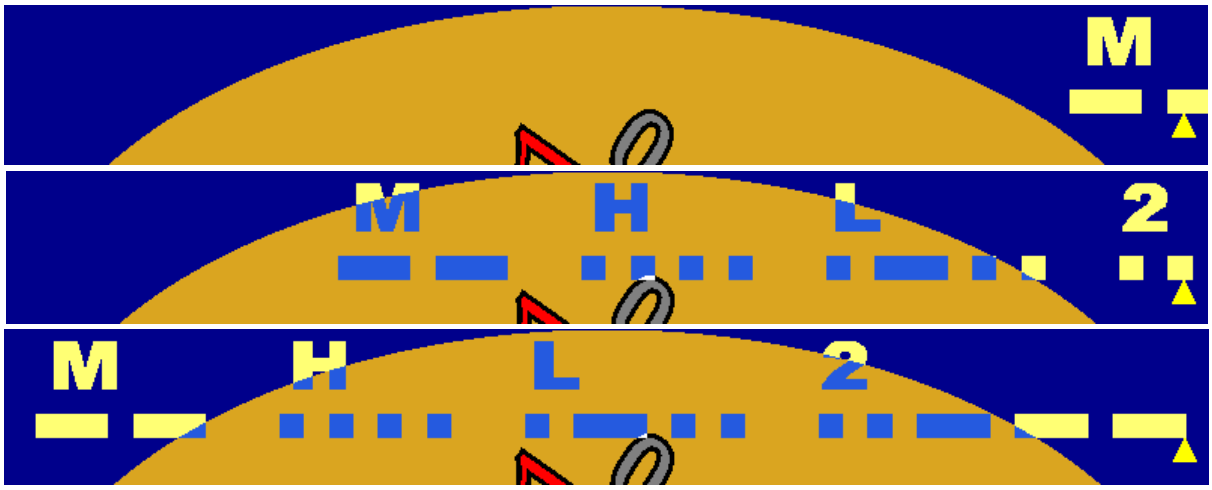
$$\mathbf{ixMorse = (x*M)/2^N} \text{ ; x in <0,799>}$$

where N and M are chosen to be as close as possible to the required mapping. The figure on the left demonstrates the possibility of gradually enlarging Morse code MHL2 only by changing the M value.



C: +5 points - Morse dot-dash with letters

To the dot-dash Morse code, add the corresponding letters that run with the code at the beginnings of their Morse patterns. Again, it is wrong to use complex expressions or place characters one by one! **Use memories**. Store the whole text, including spaces, into a single memory, and display all characters at once.



See also step F images.

D: +5 points – Control Morse speed

Control Morse transmission rate by touch and remote control. Indicate the currently set value with a suitable graphic element. Allow at least eight different speeds.

This part is more about the proper structure of your code, which we will discuss in practical exercises. Following the recommendation, the touch and remote control can be solved simultaneously without extra effort.

E: +5 points -Control waiting time

Allow the possibility of adjusting the wait time delays before repeating your Morse sequence—the delay is controlled by touch and remote control. Create a proper indicator with at least eight different values. During waiting times, a graphical indication runs signalling elapsed time. Do not program complex animations; you have no processor here! Select such elements that can be easily plugged in.

F: +10 extra points – Text path

Add displaying the numeric values of speed and wait indicators.

There are several possibilities for achieving. You can store all used texts in memory and select the proper one by its address. Eventually, you can create the digits in the style of 7-hex displays by logic.

But both methods are a little bit clumsy. Writing texts is not the domain of logic circuits. An embedded processor usually creates them in a text layer that is merged with a background image. However, our introductory course is not long enough to cover embedded processors in FPGAs. They are interesting elements but require special Qsys tools and the NIOS programming environment, significantly different from standard applications. We tried to add them once upon a time in the LSP practical exercises. After seeing the results, we left embedded processors in FPGAs for more advanced FEL courses.

We are therefore forced to stick with logic. Because of this, you will receive extra points for your logical text effort.

The following images show a possible solution for 36 points. They were taken from the simulation.

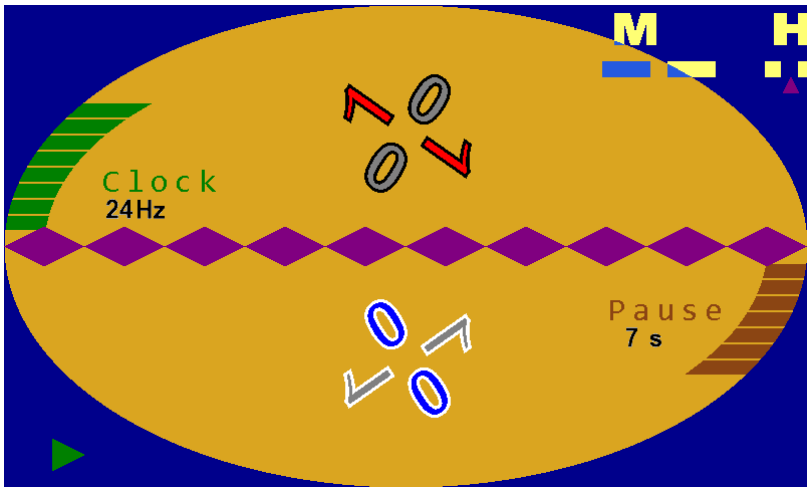


Figure 2 - 36 point solution frame 19 - the beacon is stopped

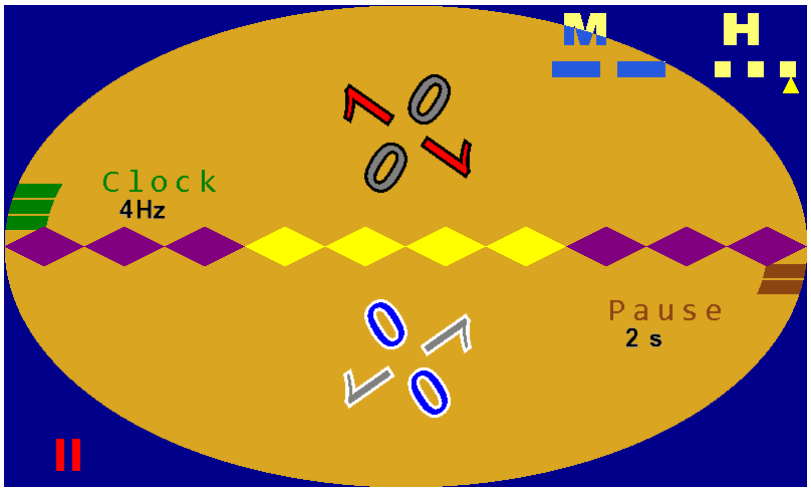


Figure 3 - 36 point solution frame 26 - the beacon runs

To speed up the simulation, the animation of the blinking element from step A was disabled. The element appears immediately, i.e., it behaves like the 1-point solution of step A.

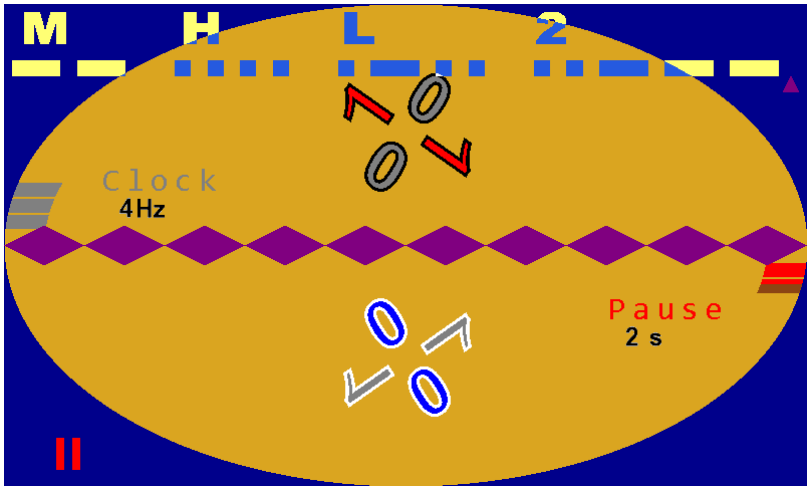


Figure 4 - 36 point solution frame 71 - the beacon waits for 2 s

Appendix: IRDA

PDF copy of page 98 of the DE2-115 User Manual, the front of the Veek-MT2.

Figure 6-18 Remote controller

Table 6-3 Key code information for each Key on remote controller

Key	Key Code	Key	Key Code	Key	Key Code	Key	Key Code
A	0x0F	B	0x13	C	0x10	⏻	0x12
1	0x01	2	0x02	3	0x03	▲	0x1A
4	0x04	5	0x05	6	0x06	▼	0x1E
7	0x07	8	0x08	9	0x09	▲	0x1B
	0x11	0	0x00	↩	0x17	▼	0x1F
▶II	0x16	◀	0x14	▶	0x18	🔇	0x0C

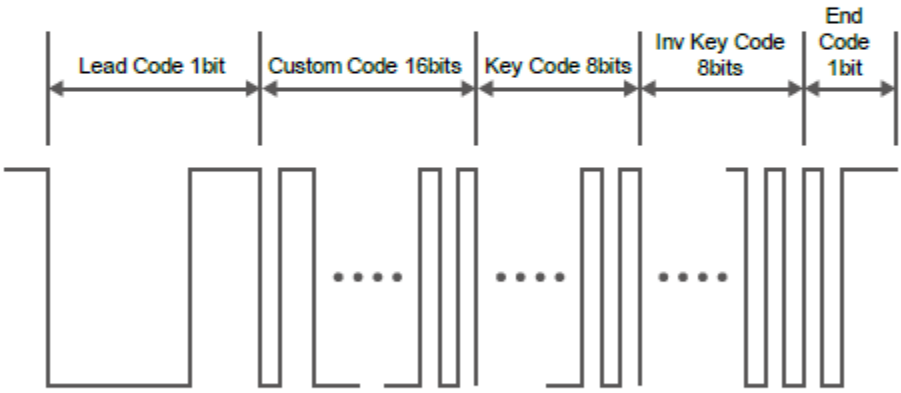


Figure 6-19 The transmitting frame of the IR remote controller